Agent Memory System

& AST Code Understanding

深度实现技术文档

Node.js Implementation with Neo4j · Tree-sitter · Vector Search · Memory Optimization

2025

1. 整体架构设计

整个系统由三大核心层组成：记忆管理层、代码理解层、以及连接两者的统一知识图谱。设计原则是让 agent 的情景记忆、语义记忆与代码结构知识共享同一张图，从而建立跨领域的关联推理能力。

架构设计原则

- 统一图谱：记忆节点与 AST 节点共存于同一图数据库，支持跨类型关联
- 分层记忆：Working Memory（内存）→ Episodic Memory（图DB）→ Semantic Memory（向量+图）
- 增量更新：AST 变更时局部更新，不全量重建，保持性能
- 异步巩固：后台定期运行记忆整合、冲突检测、衰减清理等任务

1.1 系统分层图

```
┌──────────────────────────────────────────────────────────────┐
│                      Agent Core (LLM)                        │
└──────────────────────────┬───────────────────────────────────┘
                           │ MemoryManager API
┌──────────────────────────▼───────────────────────────────────┐
│                     Memory Manager                           │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │   Working    │  │   Episodic   │  │   Semantic   │       │
│  │   Memory     │  │   Memory     │  │   Memory     │       │
│  │ (Map/Array)  │  │ (Neo4j 图)   │  │ (Vector+图)  │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│  ┌──────────────────────────────────────────────────┐        │
│  │           Memory Consolidator (后台任务)          │        │
│  │ 冲突检测 | 重复合并 | 衰减清理 | 睡眠整合          │        │
│  └──────────────────────────────────────────────────┘        │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│              Code Understanding Layer                        │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ Tree-sitter AST  │  │   Code Knowledge Graph   │          │
│  │ (增量解析引擎)    │  │ 函数/类/依赖/调用链       │          │
│  └──────────────────┘  └──────────────────────────┘          │
│  ┌──────────────────────────────────────────────────┐        │
│  │         AST Sync Engine (文件监听+增量更新)        │        │
│  └──────────────────────────────────────────────────┘        │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│              Unified Knowledge Graph (Neo4j)                 │
│     Memory Nodes ←──RELATED_TO──→ AST Nodes                  │
└──────────────────────────────────────────────────────────────┘
```

1.2 依赖包清单

包名	用途
neo4j-driver	图数据库驱动，存储记忆节点与代码结构
graphology + graphology-algorithms	内存图操作，用于轻量图计算与遍历
@xenova/transformers	本地运行嵌入模型，生成语义向量
hnswlib-node	HNSW 向量索引，高性能近似最近邻检索
tree-sitter + tree-sitter-javascript	增量 AST 解析，支持多语言
chokidar	文件系统监听，触发 AST 增量更新
node-cron	定时后台任务，执行睡眠记忆整合
uuid	生成记忆节点唯一 ID
zod	运行时数据结构验证

2. 记忆系统深度实现

2.1 记忆节点数据模型

每条记忆是图中的一个节点，包含内容、向量嵌入、元数据与状态字段。设计时需要支持后续的冲突检测、衰减计算和置信度管理。

```
// memory-schema.js
const { z } = require("zod");

const MemorySchema = z.object({
  id:          z.string().uuid(),
  type:        z.enum(["episodic","semantic","procedural","working"]),
  content:     z.string(),          // 原始文本内容
  summary:     z.string().optional(),// LLM 生成的摘要（长记忆用）
  embedding:   z.array(z.number()), // 语义向量
  confidence:  z.number().min(0).max(1).default(1.0),
  importance:  z.number().min(0).max(1).default(0.5),
  accessCount: z.number().default(0),
  lastAccessed:z.number(),           // Unix timestamp
  createdAt:   z.number(),
  source:      z.string(),           // 来源标识
  status:      z.enum(["active","conflict","deprecated","pending"]),
  tags:        z.array(z.string()).default([]),
  codeRef:     z.string().optional(),// 关联的 AST 节点 ID
  sessionId:   z.string().optional(),
});

// 记忆关系类型（图中的边）
// SIMILAR_TO    - 语义相似（相似度 > 阈值）
// CONFLICTS_WITH - 语义相近但结论矛盾
// LEADS_TO       - 时序/因果关系
// DERIVES_FROM   - 派生/推断关系
// MENTIONS_CODE  - 记忆引用某个 AST 节点
// CONSOLIDATES   - 睡眠整合后的摘要→原始记忆
```

2.2 Working Memory（工作记忆）

工作记忆是 agent 当前 session 的短期上下文，存储在内存中，容量有限。超出容量时需要执行"注意力清除"策略，将重要记忆持久化到 Episodic Memory。

```
// working-memory.js
```

```
class WorkingMemory {
  constructor(capacity = 20) {
    this.capacity = capacity;
    this.items = new Map();    // id → MemoryNode
    this.accessLog = [];       // 记录访问顺序，用于 LRU
  }

  add(memory) {
    if (this.items.size >= this.capacity) {
      this._evict();           // 容量满，触发驱逐
    }
    this.items.set(memory.id, memory);
    this.accessLog.push(memory.id);
  }

  _evict() {
    // 驱逐策略：LRU + 重要性加权
    // score = importance * 0.4 + recency * 0.4 + accessCount * 0.2
    const scored = [...this.items.values()].map(m => ({
      id: m.id,
      score: m.importance * 0.4
            + this._recencyScore(m.lastAccessed) * 0.4
            + Math.min(m.accessCount / 10, 1) * 0.2
    }));
    scored.sort((a, b) => a.score - b.score);
    const toEvict = scored.slice(0, Math.ceil(this.capacity * 0.3));
    toEvict.forEach(({ id }) => {
      this.emit("evict", this.items.get(id)); // 由 MemoryManager 持久化
      this.items.delete(id);
    });
  }

  _recencyScore(timestamp) {
    const ageMs = Date.now() - timestamp;
    const ageHours = ageMs / 3600000;
    return Math.exp(-ageHours / 24); // 24 小时半衰期
  }
}
```

2.3 Episodic Memory（情景记忆）—— Neo4j 图层

情景记忆存储在 Neo4j 中，每条记忆是一个节点，记忆之间的关系是边。这里展示核心的 CRUD 操作和图查询模式。

```
// episodic-memory.js
const neo4j = require("neo4j-driver");

class EpisodicMemory {
  constructor(uri, user, password) {
    this.driver = neo4j.driver(uri,
      neo4j.auth.basic(user, password));
```

```
}

async init() {
  const session = this.driver.session();
  // 创建索引以加速查询
  await session.run(
    "CREATE INDEX memory_id IF NOT EXISTS FOR (m:Memory) ON (m.id)"
  );
  await session.run(
    "CREATE INDEX memory_status IF NOT EXISTS FOR (m:Memory) ON (m.status)"
  );
  await session.close();
}

async save(memory) {
  const session = this.driver.session();
  try {
    await session.run(`
      MERGE (m:Memory { id: $id })
      SET m += $props
    `, {
      id: memory.id,
      props: {
        content:     memory.content,
        type:        memory.type,
        confidence:  memory.confidence,
        importance:  memory.importance,
        createdAt:   memory.createdAt,
        lastAccessed:memory.lastAccessed,
        accessCount: memory.accessCount,
        status:      memory.status,
        tags:        memory.tags,
        source:      memory.source,
      }
    });
  } finally { await session.close(); }
}

// 创建记忆关系（冲突/相似/因果）
async linkMemories(fromId, toId, relType, props = {}) {
  const session = this.driver.session();
  try {
    await session.run(`
      MATCH (a:Memory { id: $fromId })
      MATCH (b:Memory { id: $toId })
      MERGE (a)-[r:${relType}]->(b)
      SET r += $props`,
      { fromId, toId, props }
    );
  } finally { await session.close(); }
}

// 获取记忆的 K 跳邻域，用于上下文扩展
async getNeighborhood(memoryId, depth = 2) {
  const session = this.driver.session();
  try {
```

```
      const result = await session.run(`
        MATCH (start:Memory { id: $id })
        CALL apoc.path.subgraphNodes(start, {
          maxLevel: $depth,
          labelFilter: "+Memory"
        }) YIELD node
        RETURN node`,
        { id: memoryId, depth }
      );
      return result.records.map(r => r.get("node").properties);
    } finally { await session.close(); }
  }
}
```

2.4 Semantic Memory（语义记忆）—— 向量检索层

语义记忆通过向量相似度检索，结合图邻域扩展来找到相关记忆。使用 @xenova/transformers 在本地运行嵌入模型，完全不依赖外部服务。

```
// semantic-memory.js
const { pipeline } = require("@xenova/transformers");
const HNSWLib = require("hnswlib-node").HierarchicalNSW;

class SemanticMemory {
  constructor() {
    this.embedder = null;
    this.index = null;          // HNSW 向量索引
    this.idMap = new Map();     // vectorIndex → memoryId
    this.reverseMap = new Map();// memoryId → vectorIndex
    this.dim = 384;             // all-MiniLM-L6-v2 维度
    this.nextId = 0;
  }

  async init(maxElements = 100000) {
    this.embedder = await pipeline(
      "feature-extraction",
      "Xenova/all-MiniLM-L6-v2"  // 轻量高效的嵌入模型
    );
    this.index = new HNSWLib("cosine", this.dim);
    this.index.initIndex(maxElements, 16, 200, 100);
  }

  async embed(text) {
    const output = await this.embedder(text, {
      pooling: "mean", normalize: true
    });
    return Array.from(output.data);
  }

  async addMemory(memoryId, text) {
    const vector = await this.embed(text);
```

```
    const idx = this.nextId++;
    this.index.addPoint(vector, idx);
    this.idMap.set(idx, memoryId);
    this.reverseMap.set(memoryId, { idx, vector });
    return vector;
  }

  // 近似最近邻检索
  async search(query, topK = 10) {
    const queryVec = await this.embed(query);
    const { neighbors, distances } = this.index.searchKnn(
      queryVec, topK
    );
    return neighbors.map((idx, i) => ({
      memoryId: this.idMap.get(idx),
      similarity: 1 - distances[i], // cosine: distance→similarity
    }));
  }

  // 两条记忆之间的余弦相似度
  cosineSimilarity(vecA, vecB) {
    const dot = vecA.reduce((s, a, i) => s + a * vecB[i], 0);
    return dot; // 已 normalize，点积 = 余弦相似度
  }
}
```

3. 记忆优化策略详细实现

3.1 重复记忆检测与合并

新记忆写入前，先向量检索 top-K 候选，若相似度超过阈值，交由 LLM 判断是否为语义重复，若是则合并而非新增。

```
// memory-dedup.js
const DEDUP_THRESHOLD = 0.92;  // 相似度高于此值触发合并检查

class MemoryDeduplicator {
  constructor(semanticMemory, episodicMemory, llmClient) {
    this.semantic = semanticMemory;
    this.episodic = episodicMemory;
    this.llm = llmClient;
  }

  async checkAndMerge(newMemory) {
    // Step 1: 向量近邻检索
    const candidates = await this.semantic.search(
      newMemory.content, 5
    );
    const highSim = candidates.filter(c => c.similarity > DEDUP_THRESHOLD);
    if (highSim.length === 0) return { isDuplicate: false };

    // Step 2: LLM 判断语义是否真正重复（高召回，精确判断）
    for (const candidate of highSim) {
      const existing = await this.episodic.getById(candidate.memoryId);
      const isDup = await this._llmCheckDuplicate(
        newMemory.content, existing.content
      );
      if (isDup) {
        // Step 3: 执行合并
        const merged = await this._mergeMemories(existing, newMemory);
        await this.episodic.save(merged);
        return { isDuplicate: true, mergedInto: existing.id };
      }
    }
    return { isDuplicate: false };
  }

  async _llmCheckDuplicate(contentA, contentB) {
    const prompt = `判断以下两条信息是否表达相同的核心事实或知识。
      A: "${contentA}"
      B: "${contentB}"
      只回答 YES 或 NO。`;
    const res = await this.llm.complete(prompt);
    return res.trim().toUpperCase() === "YES";
  }
```

```
  async _mergeMemories(existing, incoming) {
    // 合并策略：LLM 生成语义融合版本
    const prompt = `合并以下两条相关记忆为一条更完整的记忆，保留所有关键细节：
      记忆A: "${existing.content}"
      记忆B: "${incoming.content}"`,
      只输出合并后的记忆内容，不要解释。`;
    const merged = await this.llm.complete(prompt);
    return {
      ...existing,
      content:     merged.trim(),
      confidence:  Math.max(existing.confidence, incoming.confidence),
      importance:  Math.max(existing.importance, incoming.importance),
      accessCount: existing.accessCount + incoming.accessCount,
      lastAccessed:Date.now(),
    };
  }
}
```

3.2 冲突记忆检测与处理

冲突指两条语义相近（向量距离近）但结论相反的记忆。检测逻辑：先向量召回，再用 LLM 判断是否存在语义矛盾，最后在图上创建 CONFLICTS_WITH 边并标记状态。

```
// memory-conflict.js
const CONFLICT_THRESHOLD = 0.80; // 低于 DEDUP，但足够语义相关

class ConflictDetector {
  async detect(newMemory, candidates) {
    const conflicts = [];
    for (const c of candidates) {
      if (c.similarity < CONFLICT_THRESHOLD) continue;
      const existing = await this.episodic.getById(c.memoryId);

      const result = await this._llmCheckConflict(
        newMemory.content, existing.content
      );
      if (result.isConflict) {
        conflicts.push({ existingId: existing.id, reason: result.reason });
      }
    }
    return conflicts;
  }

  async _llmCheckConflict(contentA, contentB) {
    const prompt = `分析以下两条信息是否存在直接矛盾。
      A: "${contentA}"
      B: "${contentB}"
      以 JSON 回答: { "isConflict": true/false, "reason": "原因" }`;
```

```
    const res = await this.llm.complete(prompt);
    return JSON.parse(res);
  }

  async handleConflict(newMemory, existingId, reason) {
    // 1. 新记忆标记为 conflict 状态
    newMemory.status = "conflict";
    await this.episodic.save(newMemory);

    // 2. 在图上创建冲突边（双向）
    await this.episodic.linkMemories(
      newMemory.id, existingId, "CONFLICTS_WITH",
      { detectedAt: Date.now(), reason, resolved: false }
    );

    // 3. 检索时的优先级策略：
    //    - 按 createdAt 降序（更新的记忆优先）
    //    - 同时返回 conflict 信息，让 agent 感知矛盾
    //    - confidence 更高的优先
  }

  // 冲突解消：agent 得到新证据后调用
  async resolveConflict(winnerId, loserId) {
    await this.episodic.update(loserId, {
      status: "deprecated",
      confidence: 0,
    });
    await this.episodic.updateRelation(
      winnerId, loserId, "CONFLICTS_WITH",
      { resolved: true, resolvedAt: Date.now() }
    );
  }
}
```

3.3 错误记忆修正（置信度反向传播）

当 agent 执行某个动作后得到反馈（成功或失败），需要将结果反向传播给相关记忆，更新置信度。这类似强化学习中的奖励信号。

```
// memory-correction.js
class MemoryCorrector {
  // 执行结果反馈：更新相关记忆的置信度
  async applyFeedback(actionContext, outcome) {
    // outcome: { success: boolean, errorDetails?: string }
    const relatedMemories = await this.findRelatedMemories(actionContext);

    for (const memory of relatedMemories) {
      let delta;
      if (outcome.success) {
```

```
        // 成功：置信度向 1.0 靠拢（学习率 0.1）
        delta = 0.1 * (1.0 - memory.confidence);
      } else {
        // 失败：置信度大幅下调
        delta = -0.2 * memory.confidence;
      }

      const newConf = Math.max(0, Math.min(1, memory.confidence + delta));
      await this.episodic.update(memory.id, {
        confidence: newConf,
        // 置信度过低 → 标记为待验证
        status: newConf < 0.3 ? "pending" : memory.status
      });

      // 低置信度记忆在检索时自动降权
    }
  }

  // 检索时的置信度加权（集成到 MemoryManager）
  rerank(candidates) {
    return candidates
      .map(c => ({
        ...c,
        // finalScore = similarity * confidence * (1 + importance)
        finalScore: c.similarity * c.confidence * (1 + c.importance)
      }))
      .sort((a, b) => b.finalScore - a.finalScore)
      .filter(c => c.confidence > 0.1); // 过滤极低置信度
  }
}
```

3.4 睡眠记忆整合（Sleep Consolidation）

在 agent 空闲时（类比人类睡眠），后台任务对近期工作记忆进行聚类，提取高层语义摘要，写入长期语义记忆，同时衰减低价值短期记忆。

```
// memory-consolidator.js
const cron = require("node-cron");

class SleepConsolidator {
  constructor(episodic, semantic, llm) {
    this.episodic = episodic;
    this.semantic = semantic;
    this.llm = llm;
  }

  start() {
    // 每 6 小时执行一次整合
    cron.schedule("0 */6 * * *", () => this.consolidate());
```

```
}

async consolidate() {
  console.log("[Consolidator] Starting sleep consolidation...");

  // Step 1: 获取过去 12 小时的工作记忆
  const recentMemories = await this.episodic.getRecent({
    since: Date.now() - 12 * 3600 * 1000,
    type: "episodic",
    status: "active"
  });

  if (recentMemories.length < 5) return; // 记忆太少，不值得整合

  // Step 2: 向量聚类（K-Means 或层次聚类）
  const clusters = await this._cluster(recentMemories);

  // Step 3: 对每个簇生成摘要，写入语义记忆
  for (const cluster of clusters) {
    const summary = await this._summarizeCluster(cluster);
    const summaryMemory = {
      id: uuid(),
      type: "semantic",
      content: summary,
      confidence: 0.9,
      importance: this._calcClusterImportance(cluster),
      status: "active",
      source: "consolidator",
      createdAt: Date.now(),
      lastAccessed: Date.now(),
      accessCount: 0,
      tags: ["consolidated"],
    };
    await this.episodic.save(summaryMemory);

    // 在图上连接摘要 → 原始记忆
    for (const m of cluster) {
      await this.episodic.linkMemories(
        summaryMemory.id, m.id, "CONSOLIDATES"
      );
    }
  }

  // Step 4: 记忆衰减 —— 降低低价值旧记忆的权重
  await this._decayOldMemories();
}

async _cluster(memories) {
  // 基于余弦相似度的简单贪心聚类
  const clusters = [];
  const assigned = new Set();
  for (const m of memories) {
    if (assigned.has(m.id)) continue;
```

```
      const cluster = [m];
      assigned.add(m.id);
      for (const other of memories) {
        if (assigned.has(other.id)) continue;
        const sim = this.semantic.cosineSimilarity(
          m.embedding, other.embedding
        );
        if (sim > 0.75) {
          cluster.push(other);
          assigned.add(other.id);
        }
      }
      clusters.push(cluster);
    }
    return clusters;
  }

  async _summarizeCluster(cluster) {
    const contents = cluster.map(m => `- ${m.content}`).join("\n");
    return this.llm.complete(
      `将以下相关记忆整合为一条简洁的知识摘要（一到两句话）：\n${contents}`
    );
  }

  async _decayOldMemories() {
    // 30 天前、访问次数 < 3、重要性 < 0.3 的记忆降低置信度
    const threshold = Date.now() - 30 * 24 * 3600 * 1000;
    await this.episodic.runQuery(`
      MATCH (m:Memory)
      WHERE m.createdAt < $threshold
        AND m.accessCount < 3
        AND m.importance < 0.3
        AND m.status = "active"
      SET m.confidence = m.confidence * 0.8,
          m.status = CASE WHEN m.confidence * 0.8 < 0.1
                          THEN "deprecated" ELSE m.status END
    `, { threshold });
  }
}
```

4. AST 代码理解系统

4.1 Tree-sitter 增量解析引擎

Tree-sitter 支持真正的增量解析——当文件局部修改时，只重新解析变更区域，而不是全量重建 AST。这使得实时同步成为可能。

```
// ast-parser.js
const Parser = require("tree-sitter");
const JavaScript = require("tree-sitter-javascript");
const Python = require("tree-sitter-python");

class ASTParser {
  constructor() {
    this.parsers = new Map();
    this.trees = new Map();   // filePath → Tree（保留旧树，用于增量）
    this._initParsers();
  }

  _initParsers() {
    const jsParser = new Parser();
    jsParser.setLanguage(JavaScript);
    this.parsers.set("js", jsParser);
    this.parsers.set("ts", jsParser); // 复用 JS parser for TS

    const pyParser = new Parser();
    pyParser.setLanguage(Python);
    this.parsers.set("py", pyParser);
  }

  // 首次解析（全量）
  parse(filePath, sourceCode) {
    const ext = filePath.split(".").pop();
    const parser = this.parsers.get(ext);
    if (!parser) throw new Error(`Unsupported: .${ext}`);

    const tree = parser.parse(sourceCode);
    this.trees.set(filePath, { tree, source: sourceCode });
    return tree;
  }

  // 增量解析（文件变更时调用）
  parseIncremental(filePath, newSource, changes) {
    const ext = filePath.split(".").pop();
    const parser = this.parsers.get(ext);
    const cached = this.trees.get(filePath);
    if (!cached) return this.parse(filePath, newSource);

    const oldTree = cached.tree;
```

```
    // 告知旧树哪些区域发生了变更
    // changes: [{ startIndex, oldEndIndex, newEndIndex,
    //             startPosition, oldEndPosition, newEndPosition }]
    for (const change of changes) {
      oldTree.edit(change); // 原地修改旧树的位置信息
    }

    // 增量重解析：tree-sitter 只重解析变更节点
    const newTree = parser.parse(newSource, oldTree);
    this.trees.set(filePath, { tree: newTree, source: newSource });
    return newTree;
  }

  // 提取结构信息
  extractStructure(tree, filePath) {
    const structures = [];
    const cursor = tree.walk();

    function visit(node) {
      if (node.type === "function_declaration" ||
          node.type === "method_definition" ||
          node.type === "arrow_function") {
        structures.push({
          kind: "function",
          name: node.childForFieldName("name")?.text || "<anonymous>",
          startLine: node.startPosition.row,
          endLine: node.endPosition.row,
          startIndex: node.startIndex,
          endIndex: node.endIndex,
          filePath,
        });
      } else if (node.type === "class_declaration") {
        structures.push({
          kind: "class",
          name: node.childForFieldName("name")?.text,
          startLine: node.startPosition.row,
          endLine: node.endPosition.row,
          filePath,
        });
      }
      for (const child of node.children) visit(child);
    }

    visit(tree.rootNode);
    return structures;
  }
}
```

4.2 AST 同步引擎（文件监听 + 增量更新）

通过 chokidar 监听项目文件变更，计算 diff 并触发增量解析，然后将变更的 AST 节点同步到 Neo4j 知识图谱。

```
// ast-sync-engine.js
const chokidar = require("chokidar");
const fs = require("fs").promises;

class ASTSyncEngine {
  constructor(astParser, codeGraph) {
    this.parser = astParser;
    this.graph = codeGraph;   // Neo4j code knowledge graph
    this.fileContents = new Map(); // filePath → content
  }

  watch(projectRoot) {
    const watcher = chokidar.watch(`${projectRoot}/**/*.{js,ts,py}`, {
      ignored: /(node_modules|\.git)/,
      persistent: true,
      usePolling: false,  // 使用原生事件，性能更好
    });

    watcher.on("add",    path => this._onAdd(path));
    watcher.on("change", path => this._onChange(path));
    watcher.on("unlink", path => this._onDelete(path));
    console.log(`[ASTSync] Watching ${projectRoot}`);
  }

  async _onAdd(filePath) {
    const source = await fs.readFile(filePath, "utf8");
    this.fileContents.set(filePath, source);
    const tree = this.parser.parse(filePath, source);
    const structures = this.parser.extractStructure(tree, filePath);
    await this.graph.upsertFile(filePath, structures);
  }

  async _onChange(filePath) {
    const newSource = await fs.readFile(filePath, "utf8");
    const oldSource = this.fileContents.get(filePath) || "";

    // 计算字符级别的 edit，构造 tree-sitter change 对象
    const changes = this._computeEdits(oldSource, newSource);
    this.fileContents.set(filePath, newSource);

    // 增量解析
    const newTree = this.parser.parseIncremental(
      filePath, newSource, changes
    );
    const structures = this.parser.extractStructure(newTree, filePath);

    // 增量更新图：删除消失的节点，添加新节点，更新变更节点
    await this.graph.diffUpdate(filePath, structures);
  }

  async _onDelete(filePath) {
    // 删除文件：在图上软删除（标记为 deleted）
    await this.graph.markFileDeleted(filePath);
```

```
    this.fileContents.delete(filePath);
    this.parser.trees.delete(filePath);
  }

  _computeEdits(oldSrc, newSrc) {
    // 简单实现：整文件替换
    // 生产环境建议用 diff 库计算精确的字符级别 edit
    return [{
      startIndex: 0,
      oldEndIndex: oldSrc.length,
      newEndIndex: newSrc.length,
      startPosition:   { row: 0, column: 0 },
      oldEndPosition:  this._indexToPos(oldSrc, oldSrc.length),
      newEndPosition:  this._indexToPos(newSrc, newSrc.length),
    }];
  }

  _indexToPos(source, index) {
    const lines = source.slice(0, index).split("\n");
    return { row: lines.length - 1, column: lines.at(-1).length };
  }
}
```

4.3 代码知识图谱（Code Knowledge Graph）

AST 结构存储为 Neo4j 图节点。函数、类、模块是节点；调用关系、导入关系、继承关系是边。这与记忆图共享同一数据库，从而支持"记忆 → 代码"的跨类型关联。

```
// code-graph.js
class CodeGraph {
  constructor(neo4jDriver) {
    this.driver = neo4jDriver;
  }

  async upsertFile(filePath, structures) {
    const session = this.driver.session();
    try {
      // 更新文件节点
      await session.run(`
        MERGE (f:File { path: $path })
        SET f.updatedAt = $now, f.status = "active"`,
        { path: filePath, now: Date.now() }
      );

      // 更新结构节点
      for (const s of structures) {
        const nodeId = `${filePath}:${s.kind}:${s.name}:${s.startLine}`;
        await session.run(`
          MERGE (n:CodeNode { nodeId: $nodeId })
          SET n += $props`,
          { nodeId, props: { ...s, nodeId } }
```

```
      );
      // 建立文件 → 代码节点关系
      await session.run(`
        MATCH (f:File { path: $path })
        MATCH (n:CodeNode { nodeId: $nodeId })
        MERGE (f)-[:CONTAINS]->(n)`,
        { path: filePath, nodeId }
      );
    }
  } finally { await session.close(); }
}

// 增量 diff 更新：删除失效节点，保留有效节点
async diffUpdate(filePath, newStructures) {
  const session = this.driver.session();
  try {
    const newIds = new Set(newStructures.map(s =>
      `${filePath}:${s.kind}:${s.name}:${s.startLine}`
    ));
    // 标记消失的节点为 deleted
    await session.run(`
      MATCH (f:File { path: $path })-[:CONTAINS]->(n:CodeNode)
      WHERE NOT n.nodeId IN $ids
      SET n.status = "deleted", n.deletedAt = $now`,
      { path: filePath, ids: [...newIds], now: Date.now() }
    );
    // upsert 新结构
    await this.upsertFile(filePath, newStructures);
  } finally { await session.close(); }
}

// 解析 import/require 建立依赖边
async buildDependencyEdges(filePath, imports) {
  const session = this.driver.session();
  try {
    for (const imp of imports) {
      await session.run(`
        MATCH (src:File { path: $from })
        MERGE (dep:File { path: $to })
        MERGE (src)-[:DEPENDS_ON]->(dep)`,
        { from: filePath, to: imp.resolvedPath }
      );
    }
  } finally { await session.close(); }
}

// agent 查询：找到与某段代码相关的记忆
async findMemoriesForCode(nodeId) {
  const session = this.driver.session();
  try {
    const result = await session.run(`
      MATCH (n:CodeNode { nodeId: $nodeId })<-[:MENTIONS_CODE]-(m:Memory)
      WHERE m.status = "active"
      RETURN m ORDER BY m.importance DESC LIMIT 10`,
      { nodeId }
```

```
      );
      return result.records.map(r => r.get("m").properties);
    } finally { await session.close(); }
  }
}
```

5. MemoryManager —— 统一入口

MemoryManager 是 agent 与记忆系统交互的唯一接口，封装写入管道（去重→冲突检测→持久化）和读取管道（向量检索→图扩展→重排序）。

5.1 写入管道

```
// memory-manager.js
class MemoryManager {
  constructor({ working, episodic, semantic, dedup, conflict }) {
    this.working  = working;
    this.episodic = episodic;
    this.semantic = semantic;
    this.dedup    = dedup;
    this.conflict = conflict;
  }

  // 写入管道：去重 → 冲突检测 → 向量化 → 持久化
  async remember(content, meta = {}) {
    const memory = {
      id: uuid(),
      content,
      type:        meta.type || "episodic",
      importance:  meta.importance || 0.5,
      confidence:  1.0,
      status:      "active",
      source:      meta.source || "agent",
      tags:        meta.tags || [],
      createdAt:   Date.now(),
      lastAccessed:Date.now(),
      accessCount: 0,
    };

    // Step 1: 去重检查
    const dupResult = await this.dedup.checkAndMerge(memory);
    if (dupResult.isDuplicate) {
      console.log(`[Memory] Merged into ${dupResult.mergedInto}`);
      return dupResult.mergedInto;
    }

    // Step 2: 向量化
    memory.embedding = await this.semantic.addMemory(memory.id, content);

    // Step 3: 冲突检测
    const candidates = await this.semantic.search(content, 10);
    const conflicts = await this.conflict.detect(memory, candidates);
    if (conflicts.length > 0) {
      for (const c of conflicts) {
        await this.conflict.handleConflict(memory, c.existingId, c.reason);
```

```
      }
    }

    // Step 4: 持久化到 Neo4j
    await this.episodic.save(memory);

    // Step 5: 写入工作记忆
    this.working.add(memory);

    return memory.id;
  }

  // 读取管道：向量检索 → 图扩展 → 重排序
  async recall(query, opts = {}) {
    const { topK = 10, expandDepth = 1 } = opts;

    // Step 1: 工作记忆优先（当前 session 上下文）
    const workingHits = this._searchWorking(query);

    // Step 2: 向量语义检索
    const vectorHits = await this.semantic.search(query, topK);

    // Step 3: 图邻域扩展（获取相关联的记忆）
    const expanded = [];
    for (const hit of vectorHits.slice(0, 3)) {
      const neighbors = await this.episodic.getNeighborhood(
        hit.memoryId, expandDepth
      );
      expanded.push(...neighbors);
    }

    // Step 4: 合并、去重、重排序
    const allCandidates = await this._mergeCandidates(
      workingHits, vectorHits, expanded
    );

    // Step 5: 置信度加权重排
    const reranked = this.corrector.rerank(allCandidates);

    // Step 6: 更新访问统计
    for (const m of reranked.slice(0, 5)) {
      await this.episodic.update(m.id, {
        accessCount: m.accessCount + 1,
        lastAccessed: Date.now()
      });
    }

    return reranked;
  }
}
```

5.2 Agent 使用示例

```
// agent.js 示例
const mm = new MemoryManager({ ... });

// 记忆新知识
await mm.remember("用户偏好使用 TypeScript 严格模式", {
  type: "semantic",
  importance: 0.8,
  tags: ["preference", "typescript"]
});

// 记忆并关联代码
const memId = await mm.remember("parse() 函数存在边界问题，当输入为空时崩溃", {
  type: "episodic",
  importance: 0.9,
  source: "debugging-session",
});
// 建立记忆 → 代码节点关联
await episodic.linkMemories(memId, "src/parser.js:function:parse:12",
  "MENTIONS_CODE"
);

// 检索记忆
const memories = await mm.recall("TypeScript 配置", { topK: 5 });

// 反馈修正
await mm.applyFeedback("修复 parse() 空输入", { success: true });

// 查询与某函数相关的记忆
const codeMemories = await codeGraph.findMemoriesForCode(
  "src/parser.js:function:parse:12"
);
```

6. 进阶优化策略

6.1 重要性动态计算

记忆的重要性不应是静态字段，而应根据访问频率、与其他记忆的关联数量（图中度数）、以及 LLM 判定的信息密度动态计算。

```
// importance-calculator.js
class ImportanceCalculator {
  // 综合重要性 = 0.4×访问频率 + 0.3×图连通性 + 0.3×信息密度
  async calculate(memory, graphDegree) {
    const freqScore = this._frequencyScore(memory.accessCount);
    const connScore = Math.min(graphDegree / 20, 1.0);
    const infoScore = await this._infoScore(memory.content);
    return freqScore * 0.4 + connScore * 0.3 + infoScore * 0.3;
  }

  _frequencyScore(count) {
    return 1 - Math.exp(-count / 5); // 指数增长，趋向 1.0
  }

  async _infoScore(content) {
    // 简单启发：内容越长、越具体，信息密度越高
    // 生产环境可用 LLM 评分
    const wordCount = content.split(/\s+/).length;
    return Math.min(wordCount / 50, 1.0);
  }
}
```

6.2 记忆上下文注入到 LLM

从 recall() 获取的相关记忆需要以合适的格式注入到 LLM 的 prompt 中。记忆数量过多会撑爆 context window，因此需要分层摘要压缩。

```
// context-builder.js
class ContextBuilder {
  build(memories, maxTokens = 2000) {
    // 按重要性 + 置信度排序，取 topN
    const sorted = memories
      .filter(m => m.status === "active")
      .sort((a, b) => (b.importance * b.confidence) - (a.importance * a.confidence));

    let context = "[记忆上下文]\n";
    let used = 0;
    for (const m of sorted) {
```

```
      const line = `• [${m.type}] ${m.content}` +
        (m.confidence < 0.7 ? " [低置信度]" : "") +
        (m.status === "conflict" ? " [存在冲突]" : "") + "\n";
      const tokens = line.length / 4; // 粗估
      if (used + tokens > maxTokens) break;
      context += line;
      used += tokens;
    }
    return context;
  }
}
```

6.3 AST 语义摘要给 LLM

代码理解层可以将 AST 结构转化为自然语言摘要，帮助 agent 快速理解函数的意图，而不需要读取完整源代码。

```
// code-summarizer.js
class CodeSummarizer {
  async summarizeFunction(node, sourceCode, llm) {
    const fnSource = sourceCode.slice(node.startIndex, node.endIndex);

    // 截断过长的函数
    const truncated = fnSource.length > 800
      ? fnSource.slice(0, 800) + "\n// ... (truncated)"
      : fnSource;

    const summary = await llm.complete(
      `用一句话描述以下函数的功能：\n\`\`\`\n${truncated}\n\`\`\``
    );

    // 将摘要存为与 CodeNode 关联的记忆
    await this.mm.remember(summary, {
      type: "semantic",
      importance: 0.7,
      tags: ["code-summary", node.name],
      codeRef: node.nodeId,
    });

    return summary;
  }
}
```

6.4 生产环境注意事项

性能优化

- Neo4j 事务批处理：批量写入时合并为单事务，避免每条记忆单独开事务
- 向量索引持久化：定期将 hnswlib 索引 dump 到磁盘，重启后快速恢复
- 嵌入缓存：相同文本的向量结果缓存在 Redis/内存中，避免重复推理
- AST 并发解析：多文件同时变更时，用 worker_threads 并行解析

数据安全

- 记忆加密：敏感内容（如 API keys、用户数据）存储前加密，向量化时用脱敏版本
- 访问控制：不同 sessionId 的记忆在检索时隔离，除非显式标记为 shared
- 记忆审计日志：记录所有记忆的创建、修改、删除操作，用于溯源

监控指标

- 重复率：每日新记忆中触发合并的比例（目标 < 15%）
- 冲突率：检测到冲突的比例（持续上升说明知识库不一致）
- 检索延迟：recall() 的 p99 延迟（目标 < 200ms）
- 记忆老化率：每月 deprecated 状态记忆的比例
- AST 同步延迟：文件变更到图更新完成的时间（目标 < 500ms）

6.5 推荐学习资源

资源	说明
tree-sitter 官方文档	https://tree-sitter.github.io —— 增量解析 API 详解
Neo4j Cypher 手册	https://neo4j.com/docs/cypher-manual —— 图查询语言
@xenova/transformers	https://github.com/xenova/transformers.js —— 本地嵌入
MemGPT 论文	arxiv.org/abs/2310.08560 —— 分层记忆设计参考
Cognitive Architecture for LLM	arxiv.org/abs/2309.02427 —— 记忆系统学术参考